HIGH LEVEL DESIGN (HLD)

1. System Overview

1.1 Problem Definition

Traditional customer support systems primarily rely on:

- Static FAQ pages
- Rule-based chatbots
- Human support agents

These approaches suffer from multiple limitations:

Limitations:

- Static Knowledge → Cannot adapt to new information dynamically
- Slow Response Time → Human dependency causes delays
- High Operational Cost → Requires large support teams
- Inconsistent Responses → Different agents may give different answers
- Poor User Experience → Lack of contextual understanding

Need for an Intelligent System

A modern system should:

- Understand natural language queries
- Retrieve accurate, context-relevant information
- Generate human-like responses
- Make decisions dynamically
- Escalate complex queries when needed

1.2 Objective

The objective is to design and implement an AI-powered customer support assistant using:

- **Core Capabilities:**
  - Document Understanding → Process PDF knowledge base
  - Semantic Retrieval → Use embeddings for intelligent search
  - Context-Aware Response Generation → LLM-based answers
  - Workflow Orchestration → Using LangGraph
  - Decision Intelligence → Confidence-based routing
  - Human-in-the-Loop (HITL) → Escalation mechanism

1.3 Scope of the System

In Scope:

- Single PDF ingestion and processing
- Query-response system using RAG
- Graph-based workflow execution
- Confidence-based decision-making
- HITL escalation

Out of Scope:

- Multi-user login/authentication
- Multi-document retrieval (future enhancement)
- Real-time deployment and scaling
- Integration with CRM systems

---

2. Architecture Diagram

The system is designed as a layered architecture to ensure modularity and scalability.

- **Layers Explained:**

1. User Interface Layer
   - Handles interaction with user
   - Accepts queries and displays responses

2. Document Processing Layer
   - Extracts and preprocesses text from PDF

3. Embedding & Storage Layer
   - Converts text into vector format
   - Stores embeddings in vector database

4. Retrieval Layer
   - Fetches relevant document chunks

5. LLM Processing Layer
   - Generates contextual responses

6. Workflow Orchestration Layer (LangGraph)
   - Controls execution flow
   - Enables decision-making

7. Decision & Routing Layer
   - Determines response path

8. HITL Escalation Layer
   - Handles human intervention

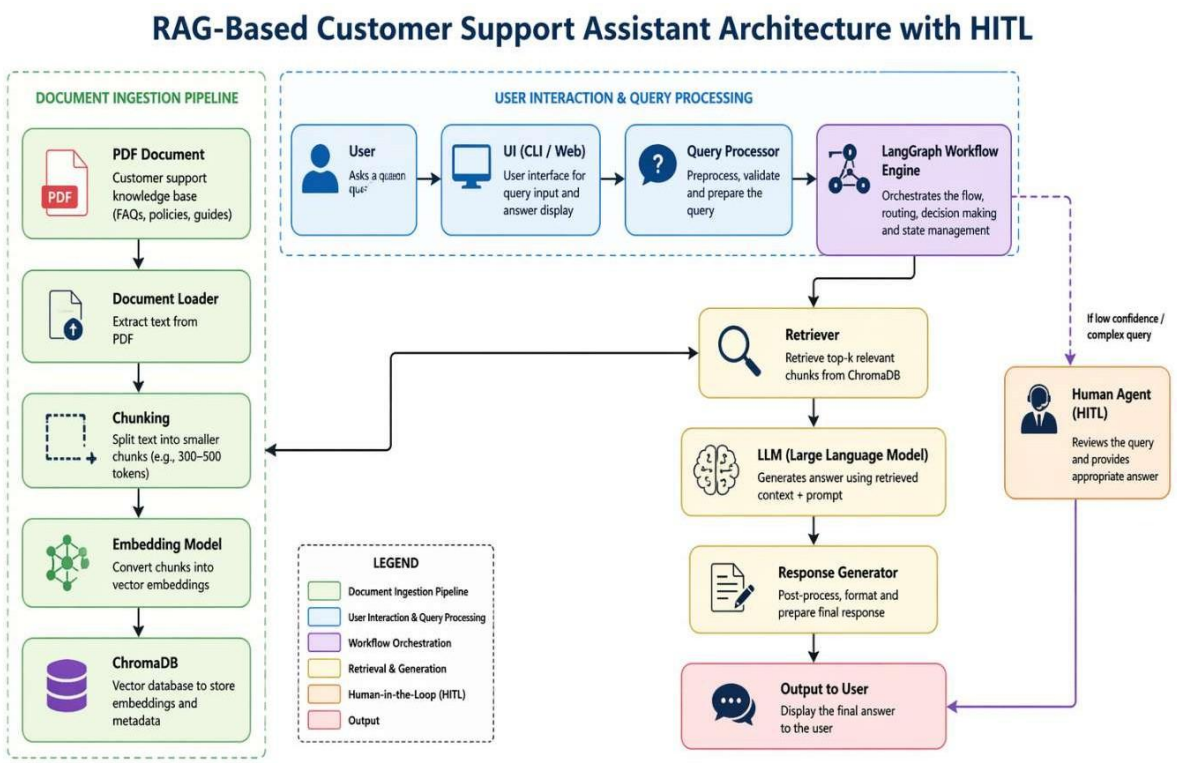

Figure: Architecture of RAG-Based Customer Support Assistant with LangGraph and HITL

Architecture Explanation:

The system consists of two main pipelines: a document processing pipeline and a query processing pipeline. In the document pipeline, the input PDF is processed through loading, chunking, and embedding stages before being stored in the ChromaDB vector database. In the query pipeline, user queries are processed through the LangGraph workflow, which orchestrates retrieval and response generation. The Retriever fetches relevant document chunks, which are passed to the LLM for generating context-aware responses. If the system detects low confidence or insufficient context, the query is escalated to the Human-in-the-Loop (HITL) module for manual intervention. This architecture ensures accurate, efficient, and reliable customer support.

---

3. <u>Component Description</u>

1. Document Loader

Loads and extracts text from PDF documents for further processing.

2. Chunking Strategy

Splits the document into smaller chunks (typically 300–500 tokens) to improve retrieval accuracy and maintain contextual relevance.

3. Embedding Model

Converts text chunks into numerical vector representations that capture semantic meaning, enabling similarity-based search.

4. Vector Store (ChromaDB)

Stores embeddings efficiently and supports fast similarity-based retrieval of relevant document chunks.

5. Retriever

Fetches top-k relevant chunks from the vector database based on the semantic similarity of the user query.

6. LLM (Large Language Model)

Generates context-aware and human-like responses using the retrieved information and user query.

7. Graph Workflow Engine (LangGraph)

Controls the system workflow, including execution flow, state management, and decision-making logic.

8. Routing Layer

Evaluates the system's confidence and determines whether to generate a response automatically or escalate the query.

9. HITL Module (Human-in-the-Loop)

Handles escalation by forwarding complex or uncertain queries to a human agent and integrating their response into the system output.

---

4. Data Flow

Document Processing Flow:

1. The PDF document is uploaded into the system.
2. Text is extracted using the Document Loader.
3. The extracted text is divided into smaller chunks.
4. Each chunk is converted into embeddings using the embedding model.
5. The embeddings are stored in the ChromaDB vector database.

Query Lifecycle

1. The user submits a query through the interface.

2. The query is processed and converted into an embedding.
3. The Retriever identifies the most relevant document chunks from ChromaDB.
4. Retrieved context is passed to the LLM.
5. The LLM generates a context-aware response.
6. The system evaluates response confidence and relevance.
7. If confidence is low or query is complex, it is escalated to the HITL module.
8. The final response (automated or human-assisted) is returned to the user.

This ensures that responses are both contextually relevant and reliable.

---

5. Technology Choices

ChromaDB

- Lightweight and efficient vector database.
- Supports fast similarity search.
- Easy integration with Python-based frameworks.
- Suitable for local and scalable deployment.

LangGraph

- Enables graph-based workflow orchestration.
- Supports conditional routing and state transitions.
- Ideal for implementing decision-driven AI systems.

LLM Choice (e.g., GPT / Llama)

- Strong natural language understanding capabilities.
- Generates accurate, context-aware, and human-like responses.
- Selected for its ability to process retrieved context effectively.

Additional Tools

- Python → Core development language
- LangChain → Simplifies RAG pipeline implementation
- Streamlit (optional) → User interface development

---

6. Scalability Considerations

Handling Large Documents

- Efficient chunking strategies to maintain context.
- Optimized indexing for faster retrieval.
- Incremental processing for large datasets.

Increasing Query Load

- Parallel query handling.
- Caching frequently asked queries.
- Scalable backend architecture.

Latency Optimization

- Limiting retrieval to top-k relevant chunks.
- Reducing prompt size for faster LLM response.
- Using efficient embedding models.

System Scalability

The system is designed to scale horizontally with increasing data volume and user demand, ensuring consistent performance.

6.1 Large Documents

- Efficient chunking
- Batch processing

6.2 High Query Load

- Parallel processing
- Caching

6.3 Latency Optimization

- Reduce LLM calls
- Optimize retrieval

6.4 Storage Scaling

- Move to distributed DB

- Multi-document support

---

7. Key Features

- Context-aware AI responses
- Efficient semantic retrieval
- Workflow-driven execution
- Intelligent decision-making
- Human fallback mechanism

---

8. Conclusion

This system represents a **modern AI architecture** that integrates:

- Retrieval-Augmented Generation (RAG)
- Graph-based workflow orchestration (LangGraph)
- Decision-making intelligence
- Human-in-the-loop escalation

It ensures:

- High accuracy
- Better reliability
- Scalable design
- Real-world applicability